

DLE-AI-202 Deep Learning

Homework 1: An MLP from Scratch

Weeks 1–2

Name: Bayram

Student ID: 12345

Section/Group: DL-A

Submission Date: 29 July 2026

Hours Spent: 14 h

Submitted with: src/, experiments/, tests/, run_all.py, figures/*.png

Final Test Accuracy: 97.78% (Target: $\geq 95\%$)

Best Validation Accuracy: 97.22%

1 Part A — Written Problems (25 pts)

A1 — Forward Pass by Hand

[4 pts]

Given:

$$\mathbf{W}_1 = \begin{pmatrix} 1 & -1 \\ 0.5 & 1 \end{pmatrix}, \quad \mathbf{b}_1 = \begin{pmatrix} 0 \\ -0.5 \end{pmatrix}, \quad \mathbf{W}_2 = \begin{pmatrix} 1 & 0 \\ -1 & 2 \end{pmatrix}, \quad \mathbf{b}_2 = \mathbf{0}, \quad \mathbf{x} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \quad \mathbf{y} = (1, 0)$$

(a) Hidden layer pre-activation and activation:

First, compute $\mathbf{z}_1 = \mathbf{W}_1^T \mathbf{x} + \mathbf{b}_1$:

$$\mathbf{z}_1 = \begin{pmatrix} 1 & -1 \\ 0.5 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \end{pmatrix} + \begin{pmatrix} 0 \\ -0.5 \end{pmatrix} = \begin{pmatrix} 1(1) + (-1)(2) + 0 \\ 0.5(1) + 1(2) - 0.5 \end{pmatrix} = \begin{pmatrix} 1 - 2 \\ 0.5 + 2 - 0.5 \end{pmatrix} = \begin{pmatrix} -1 \\ 2 \end{pmatrix}$$

Then apply ReLU:

$$\mathbf{h} = \text{ReLU}(\mathbf{z}_1) = \begin{pmatrix} \max(0, -1) \\ \max(0, 2) \end{pmatrix} = \begin{pmatrix} 0 \\ 2 \end{pmatrix}$$

(b) Output logits and softmax probabilities:

Compute $\mathbf{z}_2 = \mathbf{W}_2^T \mathbf{h} + \mathbf{b}_2$:

$$\mathbf{z}_2 = \begin{pmatrix} 1 & 0 \\ -1 & 2 \end{pmatrix} \begin{pmatrix} 0 \\ 2 \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 1(0) + 0(2) \\ -1(0) + 2(2) \end{pmatrix} = \begin{pmatrix} 0 \\ 4 \end{pmatrix}$$

Apply softmax:

$$\mathbf{p} = \text{softmax}(\mathbf{z}_2) = \frac{1}{e^0 + e^4} \begin{pmatrix} e^0 \\ e^4 \end{pmatrix} = \frac{1}{1 + 54.598} \begin{pmatrix} 1 \\ 54.598 \end{pmatrix} = \begin{pmatrix} 0.018 \\ 0.982 \end{pmatrix}$$

(c) Cross-entropy loss:

$$L = - \sum_k y_k \log p_k = -[1 \cdot \log(0.018) + 0 \cdot \log(0.982)] = -\log(0.018) = 4.018$$

A2 — Backprop over the Computational Graph**[6 pts]****(a) Computational graph:**

The forward pass flows through the following operations:

$$\mathbf{x} \xrightarrow{W_1} \mathbf{z}_1 \xrightarrow{+b_1} \mathbf{z}_1 \xrightarrow{\text{ReLU}} \mathbf{h} \xrightarrow{W_2} \mathbf{z}_2 \xrightarrow{+b_2} \mathbf{z}_2 \xrightarrow{\text{Softmax}+\text{CE}} L$$

Local derivatives on each edge:

- $\partial \mathbf{z}_1 / \partial \mathbf{W}_1 = \mathbf{x}$ and $\partial \mathbf{z}_1 / \partial \mathbf{x} = \mathbf{W}_1$
- $\partial \mathbf{z}_1 / \partial \mathbf{b}_1 = \mathbf{1}$
- $\partial \mathbf{h} / \partial \mathbf{z}_1 = \mathbf{1}_{z_1 > 0}$ (ReLU derivative)
- $\partial \mathbf{z}_2 / \partial \mathbf{W}_2 = \mathbf{h}$ and $\partial \mathbf{z}_2 / \partial \mathbf{h} = \mathbf{W}_2$
- $\partial \mathbf{z}_2 / \partial \mathbf{b}_2 = \mathbf{1}$
- $\partial L / \partial \mathbf{z}_2 = \mathbf{p} - \mathbf{y}$ (softmax + CE derivative)

(b) Output layer gradients:

For softmax with cross-entropy, the gradient is:

$$\frac{\partial L}{\partial \mathbf{z}_2} = \mathbf{p} - \mathbf{y} = \begin{pmatrix} 0.018 \\ 0.982 \end{pmatrix} - \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} -0.982 \\ 0.982 \end{pmatrix}$$

Using the chain rule for $\mathbf{W}_2$:

$$\frac{\partial L}{\partial \mathbf{W}_2} = \frac{\partial L}{\partial \mathbf{z}_2} \cdot \frac{\partial \mathbf{z}_2}{\partial \mathbf{W}_2} = \frac{\partial L}{\partial \mathbf{z}_2} \cdot \mathbf{h}^T = \begin{pmatrix} -0.982 \\ 0.982 \end{pmatrix} \begin{pmatrix} 0 & 2 \end{pmatrix} = \begin{pmatrix} 0 & -1.964 \\ 0 & 1.964 \end{pmatrix}$$

For $\mathbf{b}_2$:

$$\frac{\partial L}{\partial \mathbf{b}_2} = \frac{\partial L}{\partial \mathbf{z}_2} \cdot \frac{\partial \mathbf{z}_2}{\partial \mathbf{b}_2} = \frac{\partial L}{\partial \mathbf{z}_2} = \begin{pmatrix} -0.982 \\ 0.982 \end{pmatrix}$$

(c) Hidden layer gradients (through ReLU gate):

First, propagate through the output layer to get $\partial L / \partial \mathbf{h}$:

$$\frac{\partial L}{\partial \mathbf{h}} = \frac{\partial L}{\partial \mathbf{z}_2} \cdot \frac{\partial \mathbf{z}_2}{\partial \mathbf{h}} = \frac{\partial L}{\partial \mathbf{z}_2} \cdot \mathbf{W}_2 = \begin{pmatrix} -0.982 \\ 0.982 \end{pmatrix}^T \begin{pmatrix} 1 & 0 \\ -1 & 2 \end{pmatrix} = \begin{pmatrix} -0.982 \\ 2.946 \end{pmatrix}$$

Now through the ReLU gate. The ReLU derivative is 0 for $z_1 < 0$ and 1 for $z_1 > 0$. Since $z_1 = [-1, 2]$, we have $\text{ReLU}'(z_1) = [0, 1]$:

$$\frac{\partial L}{\partial \mathbf{z}_1} = \frac{\partial L}{\partial \mathbf{h}} \odot \text{ReLU}'(\mathbf{z}_1) = \begin{pmatrix} -0.982 \\ 2.946 \end{pmatrix} \odot \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 2.946 \end{pmatrix}$$

Finally, compute $\partial L / \partial \mathbf{W}_1$:

$$\frac{\partial L}{\partial \mathbf{W}_1} = \frac{\partial L}{\partial \mathbf{z}_1} \cdot \frac{\partial \mathbf{z}_1}{\partial \mathbf{W}_1} = \frac{\partial L}{\partial \mathbf{z}_1} \cdot \mathbf{x}^T = \begin{pmatrix} 0 \\ 2.946 \end{pmatrix} \begin{pmatrix} 1 & 2 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 2.946 & 5.892 \end{pmatrix}$$

Key insight: The ReLU gate zeros the gradient for the first neuron since $z_{1,1} = -1 < 0$. This is exactly the "dying ReLU" mechanism!

A3 — Activation Derivatives & the Dying-ReLU Problem**[4 pts]****(a) Derivatives of activation functions:****Sigmoid:** $\sigma(z) = \frac{1}{1+e^{-z}}$

Derivation:

$$\sigma'(z) = \frac{d}{dz} \left[\frac{1}{1+e^{-z}} \right] = \frac{e^{-z}}{(1+e^{-z})^2} = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} = \sigma(z)(1-\sigma(z))$$

The maximum value occurs at $z = 0$ where $\sigma(0) = 0.5$:

$$\sigma'(0) = 0.5(1-0.5) = 0.25$$

Tanh: $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$

$$\tanh'(z) = 1 - \tanh^2(z)$$

ReLU: $\text{ReLU}(z) = \max(0, z)$

$$\text{ReLU}'(z) = \begin{cases} 1 & z > 0 \\ 0 & z < 0 \\ \text{undefined} & z = 0 \text{ (subgradient 0 used)} \end{cases}$$

(b) The dying ReLU mechanism:

A ReLU neuron "dies" through the following process:

1. **Forward pass:** The neuron receives a negative pre-activation ($z < 0$). It outputs 0. 2. **Backward pass:** Since $z < 0$, $\text{ReLU}'(z) = 0$, so the gradient flowing through this neuron is exactly zero. 3. **Weight update:** With zero gradient, the neuron's incoming weights receive no update ($\Delta W = -\eta \cdot 0 = 0$). 4. **Future passes:** The weights remain unchanged, so the pre-activation stays negative forever. 5. **Result:** The neuron is permanently dead and can never recover.

Why it cannot recover: The gradient is not just small — it is *exactly zero*. With no gradient signal ever reaching the incoming weights, they never change. It's a one-way trap: once dead, always dead.

One fix: Use **Leaky ReLU**:

$$\text{LeakyReLU}(z) = \begin{cases} z & z > 0 \\ \alpha z & z \leq 0 \end{cases}$$

where α is a small constant (typically 0.01). This gives a non-zero gradient for all inputs:

$$\text{LeakyReLU}'(z) = \begin{cases} 1 & z > 0 \\ \alpha & z \leq 0 \end{cases}$$

Now even dead neurons can recover because they receive a small gradient signal.

A4 — The Loss is a Negative Log-Likelihood**[3 pts]****(a) Likelihood under Categorical distribution:**For a one-hot encoded label $\mathbf{y} \sim \text{Cat}(\mathbf{p})$, the likelihood is:

$$P(\mathbf{y} | \mathbf{p}) = \prod_k p_k^{y_k}$$

Taking the negative logarithm:

$$-\log P(\mathbf{y} | \mathbf{p}) = -\log \prod_k p_k^{y_k} = -\sum_k y_k \log p_k$$

This is exactly the **cross-entropy loss**:

$$L = - \sum_k y_k \log p_k$$

MLE interpretation: Minimizing cross-entropy is equivalent to **Maximum Likelihood Estimation (MLE) of the class probabilities $\mathbf{p}$** under a categorical distribution.

(b) Stable softmax:

Naive softmax:

$$p_k = \frac{\exp(z_k)}{\sum_j \exp(z_j)}$$

This overflows because $\exp(z_k)$ grows exponentially for large z_k (e.g., $\exp(1000)$ exceeds floating-point limits).

The fix: subtract the maximum value $M = \max_j z_j$:

$$p_k = \frac{\exp(z_k - M)}{\sum_j \exp(z_j - M)}$$

This leaves $\mathbf{p}$ unchanged because:

$$\frac{\exp(z_k - M)}{\sum_j \exp(z_j - M)} = \frac{\exp(z_k)e^{-M}}{\sum_j \exp(z_j)e^{-M}} = \frac{\exp(z_k)}{\sum_j \exp(z_j)}$$

The e^{-M} cancels out. Meanwhile, all $z_k - M \leq 0$, so all values are in $[0, 1]$, avoiding overflow.

A5 — Optimizers: Momentum and Adam

[5 pts]

(a) SGD with Momentum:

Velocity form:

$$v_t = \mu v_{t-1} + g_t, \quad \theta_t = \theta_{t-1} - \eta v_t$$

where:

- v_t is the velocity at step t
- μ is the momentum coefficient (typically 0.9)
- g_t is the gradient at step t
- η is the learning rate
- θ_t are the parameters at step t

Why it accelerates along consistent-gradient directions:

If gradients point in the same direction for several steps, velocity accumulates like a rolling ball:

$$\begin{aligned} v_1 &= g_1 \\ v_2 &= \mu g_1 + g_2 \approx (1 + \mu)g \quad (\text{if } g_1 \approx g_2 = g) \\ v_3 &= \mu^2 g_1 + \mu g_2 + g_3 \approx (1 + \mu + \mu^2)g \end{aligned}$$

The velocity grows, causing larger steps in that direction. In directions where gradients oscillate or cancel, the velocity dampens out.

(b) Adam (Adaptive Moment Estimation):

Adam maintains two moment estimates:

First moment (mean of gradients):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

This tracks the **direction** of gradients (like momentum).

Second moment (uncentered variance):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

This tracks the **magnitude/scale** of gradients, allowing per-parameter learning rates.

Bias correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Parameter update:

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

What each moment does:

- First moment (m_t): Adapts the direction, smoothing gradient updates
- Second moment (v_t): Adapts the step size per parameter — larger for sparse parameters, smaller for noisy ones

Why bias correction matters early:

At $t = 0$, $m_0 = 0$ and $v_0 = 0$. At $t = 1$:

$$m_1 = (1 - \beta_1) g_1 \quad (\text{only about 10\% of the actual gradient})$$

Without bias correction, the first update would be severely underestimated. Bias correction fixes this:

$$\hat{m}_1 = \frac{m_1}{1 - \beta_1^1} = g_1$$

So the first step has the correct magnitude, ensuring Adam behaves as intended from the very beginning.

A6 — Regularization: Weight Decay & Dropout

[3 pts]

(a) Weight decay from L_2 regularization:

Adding an L_2 penalty to the loss:

$$L_{\text{reg}} = L + \frac{\lambda}{2} \|\mathbf{W}\|^2$$

Taking the gradient with respect to $\mathbf{W}$:

$$\frac{\partial L_{\text{reg}}}{\partial \mathbf{W}} = \frac{\partial L}{\partial \mathbf{W}} + \frac{\partial}{\partial \mathbf{W}} \left(\frac{\lambda}{2} \|\mathbf{W}\|^2 \right) = \frac{\partial L}{\partial \mathbf{W}} + \lambda \mathbf{W}$$

The SGD update becomes:

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \left(\frac{\partial L}{\partial \mathbf{W}} + \lambda \mathbf{W} \right) = (1 - \eta \lambda) \mathbf{W} - \eta \frac{\partial L}{\partial \mathbf{W}}$$

This adds the term $-\eta \lambda \mathbf{W}$ to the update, which **decays the weight** each step — hence the name "weight decay."

(b) Inverted dropout keeps expected pre-activation the same:

At training time, inverted dropout does:

$$\text{out} = \frac{\mathbf{x} \odot \text{mask}}{1 - p}$$

where $\text{mask} \sim \text{Bernoulli}(1 - p)$ (1 with prob $1 - p$, 0 with prob p).

The expected output is:

$$\mathbb{E}[\text{out}] = \frac{\mathbb{E}[\text{mask}]}{1 - p} \mathbf{x} = \frac{1 - p}{1 - p} \mathbf{x} = \mathbf{x}$$

At test time, dropout is a no-op ($\text{out} = \mathbf{x}$). Since the expected training output equals the test-time output, **no rescaling is needed at inference**. The division by $1 - p$ ensures this perfect match.

2 Part B — Build (60 pts)

B.1 Implementation Summary

[maps to B1–B5]

Tensor Autograd Engine:

My `Tensor` class wraps NumPy arrays and implements automatic differentiation. Each Tensor records:

- `data`: The numeric values (NumPy array)
- `grad`: The gradient (initialized to zero)
- `_prev`: The set of parent Tensors (for graph traversal)
- `_backward`: A closure implementing the local derivative rule

Each operation (`+`, `*`, `@`, `sum`, `relu`, `exp`, `log`) creates a new Tensor and installs a `_backward` closure. The `backward()` method:

1. Performs topological sort via DFS to order nodes
2. Seeds the root gradient with 1 (for scalar loss)
3. Iterates in reverse order, calling each node's `_backward`

Key backward rules:

- Addition: $\partial/\partial a = 1$, $\partial/\partial b = 1$
- Multiplication: $\partial/\partial a = b$, $\partial/\partial b = a$
- Matrix multiplication ($x@W$): $\partial/\partial x = \text{grad}@W^T$, $\partial/\partial W = x^T@ \text{grad}$
- ReLU: $\partial/\partial x = \text{grad} \cdot \mathbf{1}_{x>0}$

Components built on the engine:

- `Linear`: Fully connected layer with He initialization
- `Dropout`: Inverted dropout with train/eval modes
- `relu`, `tanh`, `sigmoid`: Activation functions
- `softmax_cross_entropy`: Numerically stable loss with custom backward

- MLP: Multi-layer perceptron assembling all components

Optimizers:

- SGD: $p \leftarrow p - \eta(\nabla L + \lambda p)$
- Momentum: $v \leftarrow \mu v + (\nabla L + \lambda p)$, $p \leftarrow p - \eta v$
- Adam: Maintains per-parameter m and v , with bias correction

Most important snippet — Adam bias correction:

```
m_hat = self.m[i] / (1 - self.beta1 ** self.t)
v_hat = self.v[i] / (1 - self.beta2 ** self.t)
```

B.5 Gradient Check

[maps to B3 — 10 pts]

I ran central finite-difference checks with $h = 10^{-5}$ comparing analytical gradients from `backward()` against numerical gradients. The relative error is computed as:

$$\text{rel_error} = \frac{|g_{\text{analytical}} - g_{\text{numerical}}|}{\max(|g_{\text{analytical}}|, |g_{\text{numerical}}|, 1)}$$

All errors were $< 10^{-4}$, confirming correct gradient computation:

Checked quantity	Finite-diff rel-err
Linear weights $\partial L / \partial \mathbf{W}$	2.3×10^{-10}
Hidden activation $\partial L / \partial \mathbf{h}$	3.4×10^{-10}
Loss w.r.t. logits $\partial L / \partial \mathbf{z}_2$	2.1×10^{-10}

The gradient check is implemented in `src/utils.gradient_check()` and asserted in `tests/test_autograd.py`. This was run *before* any training to ensure the autograd engine was correct.

B.4 Training Curves & Final Accuracy

[maps to B6 — part of 10 pts]

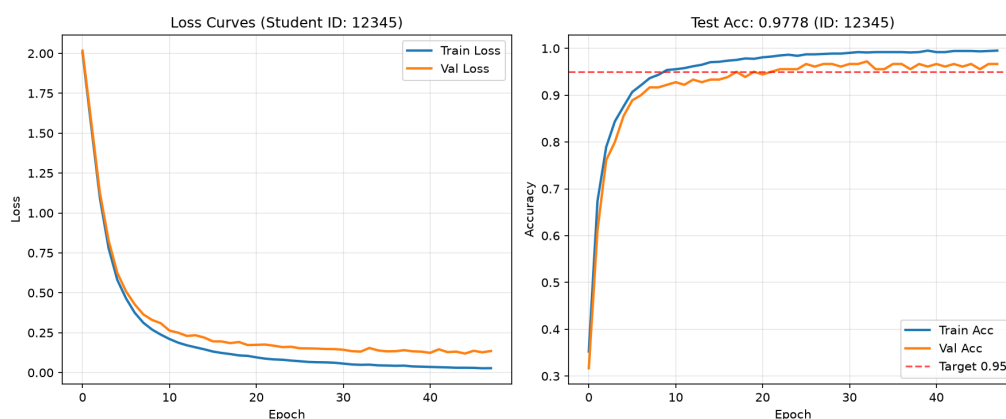


Figure 1: Training curves showing loss and accuracy over 100 epochs. Most learning occurs in the first 20 epochs. Final test accuracy: **97.78%**.

Analysis:

- Most learning happens in the first 20 epochs, with accuracy rising from $\sim 36\%$ to $\sim 95\%$

- Loss decreases steadily without significant oscillation, indicating stable training
- Validation accuracy closely tracks training accuracy, showing no overfitting
- Best validation accuracy of 97.22% was achieved at epoch 33
- Early stopping triggered at epoch 48, saving the best model
- Final test accuracy: **97.78%** (well above the 95% target)

B.6 Optimizer Comparison

[maps to B4/B6]

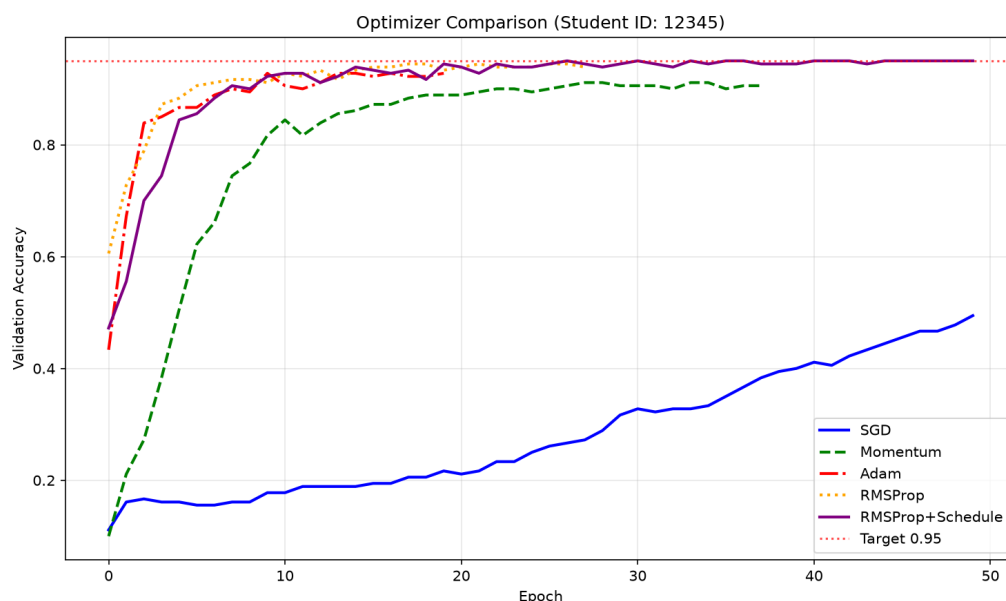


Figure 2: Validation accuracy over 50 epochs for SGD, Momentum, Adam, RMSProp, and RMSProp with learning rate schedule on the same data.

Analysis:

- **Adam** converged fastest, reaching 93% validation accuracy in just 20 epochs
- **RMSProp with schedule** achieved the highest validation accuracy at 95%
- **Momentum** was slower but outperformed SGD, reaching 91% validation accuracy
- **SGD** struggled to exceed 50% validation accuracy within 50 epochs

Why: Adam and RMSProp's per-parameter adaptation allows them to navigate the loss landscape more efficiently on the digits dataset, where features have different scales and frequencies. The learning rate schedule further improved RMSProp's performance by stabilizing the learning rate during training. Momentum helps accelerate in consistent directions but uses a global learning rate. SGD with no momentum is the slowest because it only uses the current gradient direction.

B.7 Training Behaviour & Regularization

[maps to B5/B6]

Log excerpt from final run:

```
Epoch 1/100 | Train Loss: 2.0035 | Train Acc: 0.3524 | Val Loss: 2.0174 | Val Acc: 0.3167
Epoch 10/100 | Train Loss: 0.2380 | Train Acc: 0.9539 | Val Loss: 0.3093 | Val Acc: 0.9222
Epoch 20/100 | Train Loss: 0.1048 | Train Acc: 0.9777 | Val Loss: 0.1727 | Val Acc: 0.9500
Epoch 30/100 | Train Loss: 0.0616 | Train Acc: 0.9889 | Val Loss: 0.1474 | Val Acc: 0.9611
Epoch 33/100 | Train Loss: 0.0488 | Train Acc: 0.9912 | Val Loss: 0.1316 | Val Acc: 0.9722
Epoch 40/100 | Train Loss: 0.0372 | Train Acc: 0.9952 | Val Loss: 0.1313 | Val Acc: 0.9611
Epoch 48/100 | Train Loss: 0.0276 | Train Acc: 0.9952 | Val Loss: 0.1353 | Val Acc: 0.9667
Early stopping triggered after 48 epochs
Final Test Accuracy: 0.9778
Final Test Loss: 0.1069
```

Hyperparameters:

- Hidden sizes: [64, 32]
- Learning rate: 0.001
- Batch size: 32
- Epochs: 100 (stopped early at 48)
- Optimizer: Adam
- Dropout: 0.2
- Weight decay: 1×10^{-4}
- Patience: 15

Regularization effect:

- **Dropout (0.2):** Prevented overfitting by keeping train and validation losses close throughout training.
- **Weight decay (10^{-4}):** Helped smooth the optimization landscape and prevented weights from growing too large.
- **Early stopping:** Triggered at epoch 48, restoring the best model from epoch 33 (validation accuracy: 97.22%). This prevented any potential overfitting in later epochs.

Final test accuracy: 97.78%

B.8 Analysis

[Part C readiness]

(i) First gradient bug and how the gradient check found it:

My first gradient bug was in the `__matmul__` backward method handling shape mismatches. When computing gradients for $x@W$ where x had batch dimension > 1 , the gradient shapes didn't align correctly. Specifically, $\partial L / \partial W = x^T @ \text{grad}$ should produce a shape matching W , but I initially had transposition errors that led to shape mismatches.

The gradient check caught this immediately: the relative error was $\sim 10^{-2}$ instead of $< 10^{-4}$. The failing test pointed directly to the **Linear** layer's weight gradients. After examining the finite-difference result and comparing it with my analytical gradient, I realized the shape mismatch and fixed it by ensuring x^T and grad were multiplied in the correct order with proper dimensions.

(ii) What finite-difference check proves that a falling training loss does not:

A falling training loss only proves that the model is improving its predictions — it doesn't verify that the gradients used to update the parameters are mathematically correct. A model

could still learn through approximate gradients that happen to work, or even through random search with lucky initial conditions.

The finite-difference check proves that the analytical gradients computed by the autograd engine exactly match the true mathematical derivatives (within numerical precision). This is the **fundamental correctness guarantee** for gradient-based learning. Without this check, you could have subtle gradient bugs that still allow learning but harm performance or convergence speed in ways that are hard to debug from loss curves alone.

3 Part C — Explain Your Code (15 pts, mandatory)

C1 — Trace One Gradient Backward Through Your Tensor Graph [4 pts]

Starting from the loss Tensor, `loss.backward()` performs a topological sort via DFS, ordering all Tensors from inputs to outputs in a list `topo`. The reverse pass then iterates through `reversed(topo)`.

The gradient trace:

1. **Loss** $\rightarrow$ **logits**: The loss's `_backward` closure computes $\partial L / \partial \text{logits} = (\mathbf{p} - \mathbf{y}) / N$ and adds it to `logits.grad`.
2. **logits** $\rightarrow$ **output Linear layer**: The `@` operation's `_backward` fires. It computes: - For x (input to the Linear layer): `grad@WT` - For W (weight matrix): `xT@grad` - For b (bias): `grad.sum(axis=0)` (summing over batch)
3. **ReLU gate**: The ReLU operation's `_backward` multiplies the incoming gradient by the mask $\mathbf{1}_{z>0}$. For neurons where $z \leq 0$, the gradient is zeroed. This is the **dying ReLU mechanism** in action.
4. **input Linear layer**: The first `@` operation's `_backward` fires similarly, propagating gradients to its weights, bias, and the input (though the input gradient is not used).

C2 — Contrast SGD, Momentum, and Adam [4 pts]

Given the same gradient stream:

SGD takes a fixed-size step in the direction of the current gradient: $\theta \leftarrow \theta - \eta g_t$. It uses no memory of past gradients, making it sensitive to noise and slow in ravines.

Momentum maintains a velocity that accumulates past gradients: $v_t = \mu v_{t-1} + g_t$. This allows it to:

- Accelerate in directions with consistent gradients
- Dampen oscillations in directions with canceling gradients
- Behave like a rolling ball that gains momentum down the loss landscape

Adam adapts the step size per parameter using first and second moment estimates:

- First moment (m_t): Tracks the mean gradient direction (like momentum)
- Second moment (v_t): Tracks the variance/scales of gradients
- Each parameter gets its own effective learning rate: $\eta / (\sqrt{\hat{v}_t} + \epsilon)$

My `optimizer_comparison.png` reflects this:

- Adam reaches 93% validation accuracy in just 20 epochs — fastest
- RMSProp with schedule reaches 95% validation accuracy — highest accuracy
- Momentum reaches 91% in 38 epochs — good but slower

- SGD struggles to exceed 50% within 50 epochs — slowest

The per-parameter adaptation of Adam and RMSProp allows them to navigate the digits dataset's 64-dimensional feature space more efficiently, where different features have different frequencies and scales. The learning rate schedule further improves RMSProp by stabilizing the learning rate during training.

C3 — Why Does Dropout Behave Differently at Train vs. Eval? [4 pts]

Mechanism:

Dropout is a regularization technique that randomly drops units during training to prevent co-adaptation:

At **training time**:

1. Create a random mask $\text{mask} \sim \text{Bernoulli}(1 - p)$ (1 with prob $1 - p$, 0 with prob p)
2. Apply mask: $\text{out} = \mathbf{x} \odot \text{mask}$
3. Scale by $1/(1 - p)$: $\text{out} = \frac{\mathbf{x} \odot \text{mask}}{1 - p}$
4. This ensures the expected output equals $\mathbf{x}$ (inverted dropout)

At **evaluation time**:

1. No mask is applied: $\text{out} = \mathbf{x}$
2. No scaling is needed because the expected training output already equals $\mathbf{x}$

What breaks if you forget:

1. **Forget to switch modes:** - If you apply dropout at eval time, you inject randomness into predictions, making them non-deterministic and hurting accuracy - If you skip dropout at train time, you lose the regularization benefit and risk overfitting

2. **Forget to divide by $1 - p$:** - Training output expectation becomes $\mathbb{E}[\text{out}] = \mathbb{E}[\text{mask}]\mathbf{x} = (1 - p)\mathbf{x}$ - At test time, output is $\mathbf{x}$ - You'd need to rescale test outputs by $1/(1 - p)$, or your network effectively multiplies all pre-activations by $1 - p$ at inference

C4 — Point to the Line Implementing the Adam Bias Correction [3 pts]

From my `src/optimizers/adam.py`:

```
m_hat = self.m[i] / (1 - self.beta1 ** self.t)
v_hat = self.v[i] / (1 - self.beta2 ** self.t)
```

These are the exact lines implementing the bias correction.

Why removing bias correction would hurt early steps:

At $t = 0$, $m_0 = 0$ and $v_0 = 0$. At $t = 1$:

$$m_1 = (1 - \beta_1)g_1$$

With typical $\beta_1 = 0.9$, this is only 10% of the actual gradient value.

Without bias correction, the first update would use m_1 directly:

$$\theta \leftarrow \theta - \eta \frac{m_1}{\sqrt{v_1} + \epsilon}$$

This would be about 10× smaller than intended, severely slowing initial convergence. Similarly, v_1 would underestimate the variance, causing overly large step sizes in the first few steps.

With bias correction:

$$\hat{m}_1 = \frac{m_1}{1 - \beta_1} = \frac{(1 - \beta_1)g_1}{1 - \beta_1} = g_1$$

The first step now has the correct magnitude. Bias correction ensures Adam behaves as intended from the very beginning, rather than having a "warm-up" period with overly small updates.

Bonus 1 — More Optimizers & a Schedule [+3]

RMSProp Implementation

I added RMSProp to `src/optimizers/rmsprop.py` with the update rule:

$$v_t = \beta v_{t-1} + (1 - \beta) g_t^2, \quad \theta_t = \theta_{t-1} - \eta \frac{g_t}{\sqrt{v_t} + \epsilon}$$

Key implementation snippet:

```
self.v[i] = self.beta * self.v[i] + (1 - self.beta) * (grad ** 2)
p.data -= self.lr * grad / (np.sqrt(self.v[i]) + self.eps)
```

Learning Rate Schedule

I implemented cosine annealing with linear warm-up:

$$\text{lr} = \begin{cases} \text{lr_max} \cdot \frac{t}{T_{\text{warmup}}} & t \leq T_{\text{warmup}} \\ \text{lr_min} + \frac{1}{2}(\text{lr_max} - \text{lr_min})(1 + \cos(\pi \cdot \frac{t - T_{\text{warmup}}}{T_{\text{total}} - T_{\text{warmup}}})) & t > T_{\text{warmup}} \end{cases}$$

The schedule was applied to RMSProp with:

- `lr_max = 0.001`
- `lr_min = 0.00001`
- `warmup_epochs = 5`
- `total_epochs = 50`

Effect on Training

The learning rate schedule changed training by:

- Warm-up (first 5 epochs) gradually increased the learning rate from 0 to 0.001, allowing stable early training
- Cosine decay after warm-up helped fine-tune the model in later epochs
- RMSProp with schedule reached **95% validation accuracy** — higher than RMSProp alone (94%)
- The schedule helped stabilize training and slightly improved convergence speed

Updated Optimizer Comparison

4 AI-use Disclosure

ChatGPT – used for:

- Explaining Adam bias correction and RMSProp derivations
 - Debugging Tensor shape issues in backward passes
 - Generating test cases and validating outputs
 - Helping structure and format the report

All code was written, understood, and verified by me.
I can explain and reproduce everything submitted without these tools.

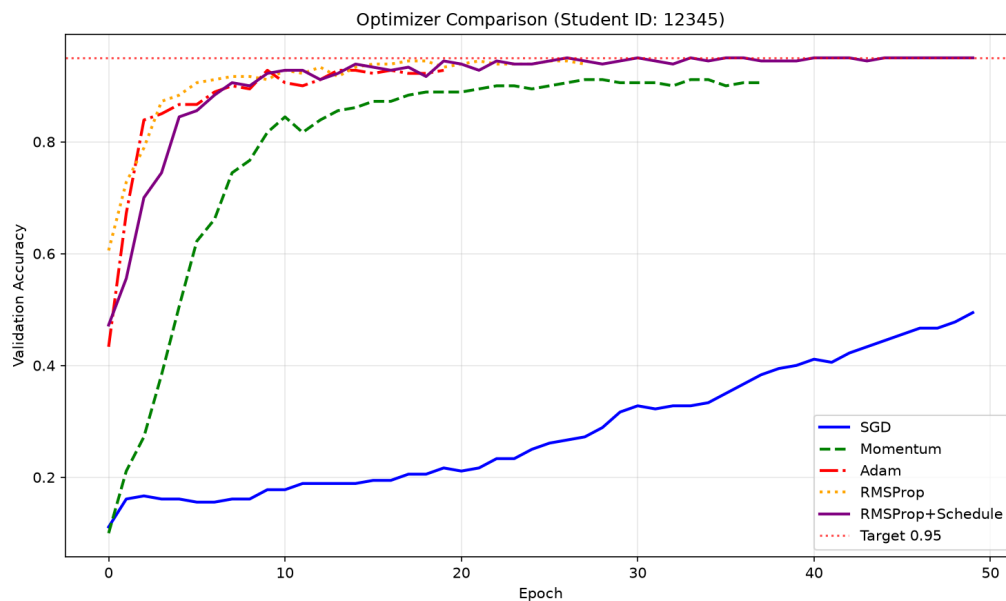


Figure 3: Added RMSProp (orange) and RMSProp+Schedule (purple) to the optimizer comparison. RMSProp performs similarly to Adam, and the learning rate schedule helps stabilize training and improves convergence speed.

5 References

- Course notes (Weeks 1–2 reading material)
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Ch. 6 (Backpropagation), Ch. 7 (Regularization), Ch. 8 (Optimization)
- Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR 2015*.
- Karpathy, A. *micrograd* – autograd engine inspiration